Assignment 1: Weirdle

Introduction to Programming (CSSE7030)

Due: 17 April 2026, 15:00 GMT+10

PUBLISHED
March 15, 2026

Changelog

- 13/03/26:
  - Initial Release.

Introduction

Wordle is an online puzzle game published by the New York Times. In a game of wordle, a word is chosen which players need to guess in a limited number of attempts. Players are given visual feedback to indicate how correct their guesses are. More detail can be found On Wikipedia, you can even play the game Online! Note that this document takes precedence if there is any difference between it and the behaviour of the above website. For our assignment we're going to make a modified version. This isn't just Wordle — it's Weirdle.

Getting Started

Download `a1.zip` from Blackboard. This archive contains the necessary files to start your assignment. Once extracted, the `a1.zip` archive will provide the following files:

- `a1.py`: *This is the only file where you write your code* and is the *only* `.py` file you will submit. *Do not* make changes to any other files.
- `support.py`: *Do not modify or submit this file*, it contains pre-defined constants to use in your assignment. In addition to these, you are encouraged to create your own constants in `a1.py` where possible.
- `words.txt`: *Do not modify or submit this file*, a dictionary of words which will be loaded and used by our game.

If you also wish to download this task sheet you must first click `View Fullscreen` at the top of the page. This will open the task sheet in a new browser tab. On this tab, use `^P` to print the page, and select `Save to PDF` as the output location. The print button on Blackboard

only captures the first two or so pages of this sheet (a limitation of Blackboard beyond our control).

> **Caution**
>
> The task sheet may be modified throughout the course of the assignment as issues are identified. These updates will appear online, but will **not** be reflected in a previously downloaded copy. Please regularly check the changelog **online** to ensure you are working with the most up to date version of the task sheet.

Structure of this Document

In programming, it is common to start with a high-level description of a program and then break it down into manageable tasks. In the spirit of this tradition, we are providing a high-level description in the Gameplay section, and a list of actionable tasks in the Implementation section. You are welcome (and encouraged) to read the Gameplay section, however bear in mind that the Implementation section lists the actual tasks you will need to complete for this assignment.

Gameplay

This section provides a high-level overview of gameplay, and is intended to provide you with an idea of the behaviour of a fully completed assignment. Where interactions are not explicitly mentioned in this section, please see the Implementation section.

> **Important**
>
> Do not simply implement the behaviour described here using your own structure; you *must* implement the individual functions described in the Implementation section. Inputs and outputs must match *exactly* what is expected. Refer to the Implementation section, the given examples, and the provided Gradescope tests for clarification on required prompts and outputs.

The state of the game is represented with two data structures: a board and a keyboard. The board is a list of entries, where each entry stores the guess and its corresponding feedback. The purpose of the keyboard is to keep a history of the "status" of each letter. In this way the player has two ways to see their guessing progress. The game represents status of letters with the following symbols:

Table 1: Symbols used by the game to represent guess statuses

Symbol	Meaning

`–`	Empty: no status
`B`	Black: : Letter is not in the secret word.
`Y`	Yellow: Letter is in the secret word, but has been guessed in the wrong position.
`G`	Green: Letter is in the secret word, and has been guessed in the correct position.

The game has two phases, a setup phase and a play phase. During the setup phase:

1. A (friendly) welcome message is shown.
2. The desired number of guesses is determined.
3. The game state (secret word, board, keyboard etc.) is created.

The game then proceeds to the gameplay phase, during which:

1. The current game state is displayed.
2. The player is prompted to give input.
3. Input can be either a special command or a guess command. The command types, their validity rules and their corresponding actions are summarised in Table 2
4. If the player entered a valid command, then action is taken according to Table 2. Otherwise the player is informed their command was invalid and gameplay returns to step 2.
5. The game state is checked. If a player has won, the state of the game and a message are displayed announcing their victory, and the game immediately proceeds to Step 6. If the player has no guesses left, the game is lost, a message is displayed and the game then proceeds to step 6. If the game is not over, gameplay returns to step 2.
6. A prompt is displayed asking whether another game is desired.
7. If the user enters `Y` or `y`, the game is reset as described in the setup phase steps and a new game begins, otherwise the program terminates gracefully.

Table 2: Valid commands and the actions that should be taken. If the command entered by the user does not *exactly* match one of the commands in this table then no action should be taken, the player should be informed their command was invalid, and the program should return immediately to step 3.

Command Type	Validity Rules	Action
help	Command is `h` or `H`	Display a help message, then proceed to step 2.

keyboard	Command is `a` or `A`	Display the state of the keyboard, then proceed to step 2.
quit	Command is `q` or `Q`	End the game, then proceed to step 6.
guess	Command has the correct length. Command contains only lowercase letters. All letters in command are unique. Command is one of the words in the game's vocabulary.	The board is updated, then the keyboard is updated, then the board is displayed. Proceed to step 5.

Implementation

This assessment has been designed to allow you to practice what you have learnt in this course so far. As such, you *must only use* the functions, operators and data types presented to you in lectures up to (and including) Week 5. Namely, the following techniques are permitted for use in this assignment:

- Functions (`def`, `return`)
- Basic control structures (`for`, `while`, `if`/`elif`/`else`, `break`)
- Primitive data types (`int`, `float`, `str`, `bool`) and their methods
- Type casting
- Variable assignment (`=`)
- Arithmetic (`+`, `-`, `*`, `\`, `\\`, `%` etc.)
- Comparison (`==`, `<=`, `>=`, `<`, `>`, `!=` etc.)
- Basic Logic (`not`, `and`, `or` etc.)
- lists and tuples, and their methods
- dictionaries and their methods
- `range` and `enumerate`
- `input` and `print`

Using any functions, operators and data types that have not been presented to you in lectures up to (and including) Week 5 will result in a deduction of up to 100% of your mark.

A pinned thread will be maintained on the Edstem discussion board with a list of permitted techniques. If you would like clarification on whether you are permitted to use a specific technique, please first check this list. If the technique has not been mentioned, please ask about permission to use the technique in a comment on this pinned thread.

> **Important**
>
> You are **not** permitted to add any additional import statements to `a1.py`. Doing so will result in a deduction of up to 100% of your mark. You must not modify or remove the import statements already provided to you in `a1.py`. Removing or modifying these existing import statements may result in your code not functioning, and may result in a deduction of up to 100% of your mark.

Required Functions

This section outlines the functions you are **required** to implement in `a1.py`. Your program must operate *exactly* as specified. In particular, your program's output must match *exactly* with the expected output. Your program will be marked automatically so minor differences in output (such as whitespace or casing) *will* cause tests to fail resulting in *zero marks* for that test.

> **Note**
>
> You are awarded marks for the number of tests passed by your functions when they are tested *independently* of one another. Thus an incomplete assignment with *some* working functions may well be awarded more marks than a complete assignment with faulty functions.

Each function is accompanied with some examples for usage to help you *start* your own testing. You should also test your functions with other values to ensure they operate according to the descriptions.

The following functions **must** be implemented in `a1.py`. They have been listed in a rough order of increasing difficulty. This does not mean that earlier functions are necessarily worth less marks than later functions. It is *highly recommended* that you do not begin work on a later function until each of the preceding functions can *at least* behave as per the shown examples. You may implement additional functions if you think they will help with your logic or make your code easier to understand.

You should be comfortable with course content up to and including the introduction of functions before commencing.

Task 1 `num_hours() -> float`

This function should return the number of hours (as a *float*) that you have spent on the assignment. The purpose of this function is to enable you to verify that you understand how to submit to Gradescope as soon as possible (see Assignment Submission), and to allow us to

gauge difficulty level of this assignment in order to provide the best possible assistance. You will not be marked differently for spending more or less time on the assignment.

If the Gradescope tests have been released, you must ensure this function passes the relevant test before seeking help regarding Gradescope issues for any of the later functions. The test will only ensure you have created a function with the correct name and number of arguments, which returns a float and does not prompt for input. You will not be marked incorrect for returning the 'wrong' number of hours.

Example:

```
>>> num_hours()
```

```
3.1415962
```

Task 2 `has_won(guess: str, target: str) -> bool`

Returns True if the guess matches the target, False otherwise.

Example:

```
>>> has_won("password", "password")
```

```
True
```

```
>>> has_won("Password", "password")
```

```
False
```

```
>>> has_won("p4ssword", "password")
```

```
False
```

```
>>> has_won("This word is WeiRd!!!111!!!", "This word is W
```

```
True
```

Task 3 `get_max_guesses() -> int`

Prompt the user to enter the desired number of guesses for a game. The guess number must be a whole number between 5 and 9 (inclusive). The function should keep prompting the user until a valid number is entered.

Example:

```
>>> get_max_guesses()
```

```
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 6
6
```

```
>>> get_max_guesses()
```

```
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 5.0
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 5
5
```

```
>>> get_max_guesses()
```

```
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: seven
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 7
7
```

```
>>> get_max_guesses()
```

```
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 12
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 10
Please enter the number of guesses you require to guess
the secret word. Your input must be a number between 5
and 9: 9
9
```

Task 4 `create_board(max_guesses: int) -> list[tuple[str, str]]`

Generates an empty board to support the given number of guesses. The generated board will hold "empty" entries (See the `EMPTY` constant in `support.py`).

A board is represented by a list containing tuples of strings. The first

string in each tuple represents a guess which the player has made, the second string represents corresponding feedback for that guess.

Preconditions:

- `board_size` will not be negative

> **Note**
> In this and future examples, our assignment rendering software may display each row of the board on a new line. IDLE is unlikely to do this by default, but that is OK; it will not impact your grade.

Example:

```
>>> create_board(0)
```

```
[]
```

```
>>> create_board(1)
```

```
[('------', '------')]
```

```
>>> create_board(6)
```

```
[('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------')]
```

```
>>> create_board(10)
```

```
[('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------'),
 ('------', '------')]
```

Task 5 `display_board(board: list[tuple[str, str]]) -> None`

Prints the given board to the screen in a visually appealing format. See the below examples for the correct format. Note that Gradescope requires your output to match *exactly* (including whitespace). You can assume that all entries in the board have exactly six characters.

> **Note**

> **Tip**
>
> This function does not return anything.

Example:

```
>>> board = create_board(5)
>>> display_board(board)
```

```
-----------------------
Guess 1:  ------
Feedback: ------
-----------------------
Guess 2:  ------
Feedback: ------
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> board = [("planet", "GYBYYB"), ("pencil", "GGGGGG"), (
>>> display_board(board)
```

```
-----------------------
Guess 1:  planet
Feedback: GYBYYB
-----------------------
Guess 2:  pencil
Feedback: GGGGGG
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
```

```
>>> board = [("random", "YYYYGB"), ("dragon", "GGGGGG")]
>>> display_board(board)
```

```
-----------------------
```

```
Guess 1:  random
Feedback: YYYYGB
------------------------
Guess 2:  dragon
Feedback: GGGGGG
------------------------
```

Task 6 `generate_secret_word() -> str`

Randomly choose a secret word from the collection of words loaded at startup.

> **Important**
>
> This function must call randint once and only once.

Example:

```
>>> generate_secret_word()
```

```
'pencil'
```

```
>>> generate_secret_word()
```

```
'dragon'
```

```
>>> generate_secret_word()
```

```
'bright'
```

Task 7 `validate_input(command: str) -> bool`

Returns a boolean that is True if (and only if) the given command is of a valid format. Possible valid command formats are given in Table 2.

Example:

```
>>> validate_input("h")
```

```
True
```

```
>>> validate_input("H")
```

```
True
```

```
>>> validate_input("Help")
```

```
Invalid guess! Guess must be 6 alphabetic characters

False
```

```
>>> validate_input("pencil")
```

```
True
```

```
>>> validate_input("Pencil")
```

```
Invalid guess! Guess must consist of unique lowercase
characters

False
```

```
>>> validate_input("penccl")
```

```
Invalid guess! Guess must consist of unique lowercase
characters

False
```

```
>>> validate_input("pen123")
```

```
Invalid guess! Guess must be 6 alphabetic characters

False
```

```
>>> validate_input("qQ")
```

```
Invalid guess! Guess must be 6 alphabetic characters

False
```

```
>>> validate_input("hq")
```

```
Invalid guess! Guess must be 6 alphabetic characters

False
```

```
>>> validate_input("")
```

```
Invalid guess! Guess must be 6 alphabetic characters

False
```

Task 8 `get_command() -> str`

Repeatedly prompts the user until they enter a valid command, at which point that command is returned. If the given command is not valid, `ENTER_COMMAND_MESSAGE` (given in `support.py`) will be printed to the screen. The possible valid commands are given in Table 2.

Example:

```
>>> get_command()
```

```
Please enter your guess (h to see valid format): Help
Invalid guess! Guess must be 6 alphabetic characters
Please enter your guess (h to see valid format): H
'H'
```

```
>>> get_command()
```

```
Please enter your guess (h to see valid format): bright
'bright'
```

```
>>> get_command()
```

```
Please enter your guess (h to see valid format): brie
Invalid guess! Guess must be 6 alphabetic characters
Please enter your guess (h to see valid format): briiht
Invalid guess! Guess must consist of unique lowercase
characters
Please enter your guess (h to see valid format): 123456
Invalid guess! Guess must be 6 alphabetic characters
Please enter your guess (h to see valid format): police
Invalid guess! Guess must be a word from the given file
Please enter your guess (h to see valid format): planet
'planet'
```

Task 9 `get_feedback(guess: str, target: str) -> str`

Give feedback indicating how closely a guess matches a target word. Symbols and their meaning are described in Table 1. Both guess and target will be the same length as `GUESS_LENGTH`, as defined in `support.py`. Both guess and target will contain unique characters.

Example:

```
>>> get_feedback("forest", "bright")
```

```
'BBYBBG'
```

```
>>> get_feedback("rocket", "rocket")
```

```
'GGGGGG'
```

```
>>> get_feedback("123456", "joking")
```

```
'BBBBBB'
```

```
>>> get_feedback("123456", "132876")
```

```
'GYYBBG'
```

Task 10 `update_board(board: list[tuple], guess_num: int, guess: str, target: str) -> None`

Update a game board to incorporate a new guess and its corresponding feedback. This function must not return anything.

> **Tip**
>
> This function must call `get_feedback`

Example:

```
>>> board = create_board(5)
>>> display_board(board)
```

```
-----------------------
Guess 1:  ------
Feedback: ------
-----------------------
Guess 2:  ------
Feedback: ------
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
```

```
-----------------------
```

```
>>> guess_num = 1
>>> guess = "planet"
>>> target = "pencil"
>>> update_board(board, guess_num, guess, target)
>>> display_board(board)
```

```
-----------------------
Guess 1:  planet
Feedback: GYBYYB
-----------------------
Guess 2:  ------
Feedback: ------
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> guess_num = 2
>>> guess = "flight"
>>> target = "flight"
>>> update_board(board, guess_num, guess, target)
>>> display_board(board)
```

```
-----------------------
Guess 1:  planet
Feedback: GYBYYB
-----------------------
Guess 2:  flight
Feedback: GGGGGG
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> guess_num = 3
>>> guess = "123456"
>>> target = "0789ab"
```

```
>>> update_board(board, guess_num, guess, target)
>>> display_board(board)
```

```
-----------------------
Guess 1:  planet
Feedback: GYBYYB
-----------------------
Guess 2:  flight
Feedback: GGGGGG
-----------------------
Guess 3:  123456
Feedback: BBBBBB
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> guess_num = 1
>>> guess = "123456"
>>> target = "123456"
>>> update_board(board, guess_num, guess, target)
>>> display_board(board)
```

```
-----------------------
Guess 1:  123456
Feedback: GGGGGG
-----------------------
Guess 2:  flight
Feedback: GGGGGG
-----------------------
Guess 3:  123456
Feedback: BBBBBB
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

Task 11 `display_keyboard(keyboard: dict[str, str]) -> None`

Prints the given keyboard to the screen in a visually appealing format. See the below examples for the correct format. Note that Gradescope requires your output to match *exactly* (including whitespace).

> **Caution**

You must use `create_keyboard` from `support.py` to make a keyboard before displaying it.

Tip

This function does not return anything.

Example:

```
>>> keyboard = create_keyboard()
>>> display_keyboard(keyboard)
```

```
Keyboard:
------------------------
a: -    b: -    c: -
d: -    e: -    f: -
g: -    h: -    i: -
j: -    k: -    l: -
m: -    n: -    o: -
p: -    q: -    r: -
s: -    t: -    u: -
v: -    w: -    x: -
y: -    z: -
------------------------
```

```
>>> keyboard = create_keyboard()
>>> keyboard["p"] = "G"
>>> keyboard["l"] = "Y"
>>> keyboard["a"] = "B"
>>> keyboard["n"] = "Y"
>>> keyboard["e"] = "Y"
>>> keyboard["t"] = "B"
>>> display_keyboard(keyboard)
```

```
Keyboard:
------------------------
a: B    b: -    c: -
d: -    e: Y    f: -
g: -    h: -    i: -
j: -    k: -    l: Y
m: -    n: Y    o: -
p: G    q: -    r: -
s: -    t: B    u: -
v: -    w: -    x: -
y: -    z: -
------------------------
```

```
>>> dodgy = {
    1: 2,
    "three": 4.0,
```

```
    "why": "not",
    "try": "this?"}
>>> display_keyboard(dodgy)
```

```
Keyboard:
-----------------------
1: 2    three: 4.0    why: not
try: this?
-----------------------
```

Task 12 `update_keyboard(board: list[tuple], keyboard: dict[str, str], guess_num: int) -> None`

Update a keyboard to incorporate a board's entry. Letters with green feedback remain green and cannot have their status changed. This function must not return anything.

Example:

```
>>> board = create_board(5)
>>> keyboard = create_keyboard()
>>> display_board(board)
```

```
-----------------------
Guess 1:  ------
Feedback: ------
-----------------------
Guess 2:  ------
Feedback: ------
-----------------------
Guess 3:  ------
Feedback: ------
-----------------------
Guess 4:  ------
Feedback: ------
-----------------------
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> display_keyboard(keyboard)
```

```
Keyboard:
-----------------------
a: -    b: -    c: -
d: -    e: -    f: -
g: -    h: -    i: -
j: -    k: -    l: -
m: -    n: -    o: -
p: -    q: -    r: -
s: -    t: -    u: -
```

```
v: -    w: -    x: -
y: -    z: -
------------------------
```

```
>>> guess_num = 1
>>> guess = "planet"
>>> target = "pencil"
>>> update_board(board, guess_num, guess, target)
>>> update_keyboard(board, keyboard, guess_num)
>>> display_board(board)
```

```
------------------------
Guess 1:  planet
Feedback: GYBYYB
------------------------
Guess 2:  ------
Feedback: ------
------------------------
Guess 3:  ------
Feedback: ------
------------------------
Guess 4:  ------
Feedback: ------
------------------------
Guess 5:  ------
Feedback: ------
------------------------
```

```
>>> display_keyboard(keyboard)
```

```
Keyboard:
------------------------
a: B    b: -    c: -
d: -    e: Y    f: -
g: -    h: -    i: -
j: -    k: -    l: Y
m: -    n: Y    o: -
p: G    q: -    r: -
s: -    t: B    u: -
v: -    w: -    x: -
y: -    z: -
------------------------
```

```
>>> guess_num = 2
>>> guess = "policy"
>>> target = "flight"
>>> update_board(board, guess_num, guess, target)
>>> update_keyboard(board, keyboard, guess_num)
>>> display_board(board)
```

```
------------------------
```

```
Guess 1:  planet
Feedback: GYBYYB
------------------------
Guess 2:  policy
Feedback: BBYYBB
------------------------
Guess 3:  ------
Feedback: ------
------------------------
Guess 4:  ------
Feedback: ------
------------------------
Guess 5:  ------
Feedback: ------
------------------------
```

```
>>> display_keyboard(keyboard)
```

```
Keyboard:
------------------------
a: B    b: -    c: B
d: -    e: Y    f: -
g: -    h: -    i: Y
j: -    k: -    l: Y
m: -    n: Y    o: B
p: G    q: -    r: -
s: -    t: B    u: -
v: -    w: -    x: -
y: B    z: -
------------------------
```

```
>>> guess_num = 3
>>> guess = "dragon"
>>> target = "dragon"
>>> update_board(board, guess_num, guess, target)
>>> update_keyboard(board, keyboard, guess_num)
>>> display_board(board)
```

```
------------------------
Guess 1:  planet
Feedback: GYBYYB
------------------------
Guess 2:  policy
Feedback: BBYYBB
------------------------
Guess 3:  dragon
Feedback: GGGGGG
------------------------
Guess 4:  ------
Feedback: ------
------------------------
```

```
Guess 5:  ------
Feedback: ------
-----------------------
```

```
>>> display_keyboard(keyboard)
```

```
Keyboard:
-----------------------
a: G    b: -    c: B
d: G    e: Y    f: -
g: G    h: -    i: Y
j: -    k: -    l: Y
m: -    n: G    o: G
p: G    q: -    r: G
s: -    t: B    u: -
v: -    w: -    x: -
y: B    z: -
-----------------------
```

Task 13 play_game() -> None:

Coordinates gameplay of a single game of *Weirdle* from start to finish according to the Gameplay section (excluding steps 6 and 7). The print output from your `play_game` function (including prompts) must exactly match the expected output. In order to help you match output exactly, most messages that should be printed during gameplay are provided as constants in `support.py`. Running the Gradescope tests will give you a good idea of whether your prompts and other outputs are correct.

Task 14 main() -> None:

The main function should be called when the file is run. The main function handles steps 6 and 7 from the Gameplay section. It repeatedly launches a game of *Weirdle* using the `play_game` function until the user specifies they no longer wish to continue. `RETRY_MESSAGE` (given in `support.py`) provides the prompt you should use when asking the user to play again.

The `if __name__ == "__main__":` block will ensure that the code in the `main` function is run when your `a1.py` file is run. Do not call your `main` function outside of this block, and do not call any other function outside this block unless you are calling them from within the body of another function.

Assessment and Marking Criteria

This assignment assesses following course learning objectives:

1. Apply program constructs such as variables, selection, iteration

and sub-routines,
2. Read and analyse code written by others,
3. Read and analyse a design and be able to translate the design into a working program, and
4. Apply techniques for testing and debugging.

There are a total of 100 marks for this assessment item.

Functionality

Your program's functionality will be marked out of a total of 60 marks.

Your assignment will be put through a series of tests and your functionality mark will be proportional to the number of tests you pass. You will be given a *subset* of the functionality tests before the due date for the assignment.

You may receive partial marks for partially working functions, or for implementing only a few functions.

> **Warning**
>
> You need to perform your *own* testing of your program to make sure that it meets **all** specifications given in the assignment. Only relying on the provided tests is likely to result in your program failing in some cases and you losing some functionality marks.

> **Note**
>
> Functionality tests are automated, so string outputs need to match *exactly* what is expected.

Your program must run in Gradescope, which uses Python 3.12. Partial solutions will be marked but if there are errors in your code that cause the interpreter to fail to execute your program, you will get zero for functionality marks. If there is a part of your code that causes the interpreter to fail, comment out the code so that the remainder can run. Your program must run using the Python 3.12 interpreter. If it runs in another environment (e.g. Python 3.8 or PyCharm) but not in the Python 3.12 interpreter, you will get zero for the functionality mark. Python versions newer than 3.12 will function correctly provided you do not use language features that are not introduced in the course.

Code Style

The style of your assignment will be assessed by a tutor. Style will be marked according to the style rubric provided with the assignment. The style mark will be out of 40.

You are expected to follow the PEP-8 style guidelines discussed in lectures. The key consideration in marking your code style is whether the code is easy to understand. There are several aspects of code style that contribute to how easy it is to understand code. In this assignment, your code style will be assessed against the following criteria:

- Readability
  - Program Structure: Layout of code makes it easy to read and follow its logic. This includes using whitespace to highlight blocks of logic, and ensuring all lines are below 80 characters.
  - Descriptive Identifier Names: Variable, constant, and function names clearly describe what they represent in the program's logic. Do not use Hungarian Notation for identifiers. In short, this means do not include the identifier's type in its name, rather make the name meaningful (e.g. employee identifier).
  - Named Constants: Any non-trivial fixed value (literal constant) in the code is represented by a descriptive named constant (identifier).
- Algorithmic Logic
  - Single Instance of Logic: Blocks of code should not be duplicated in your program. Any code that needs to be used multiple times should be implemented as a function.
  - Variable Scope: Variables should be declared locally in the function in which they are needed. Global variables should not be used.
  - Control Structures: Logic is structured simply and clearly through good use of control structures (e.g. loops and conditional statements).

- Documentation:
  - Comment Clarity: Comments provide meaningful descriptions of the code. They should not repeat what is already obvious by reading the code (e.g. `# Setting variable to 0`). Comments should not be verbose or excessive, as this can make it difficult to follow the code. Comments must be written in English.
  - Informative Docstrings: Every function should have a docstring that summarises its purpose. This includes describing parameters and return values (including type information) so that others can understand how to use the function correctly.
  - Description of Logic: All significant blocks of code should have a comment (in English) to explain how the logic works. For a small function, this would usually be the docstring. For

long or complex functions, there may be different blocks of code in the function. Each of these should have an in-line comment describing the logic.

Assignment Submission

You must submit your assignment electronically via Gradescope. You **must** use your UQ email address which is based on your student number (e.g. `s4123456@student.uq.edu.au`) as your Gradescope submission account.

When you login to Gradescope you may be presented with a list of courses. Select `CSSE7030`. You will see a list of assignments. Choose `Assignment 1`. You will be prompted to choose a file to upload. Do *not* select the option to submit using a git repository if it is presented to you. The prompt may say that you can upload any files, including zip files. You **must** submit your assignment as a single Python file called `a1.py` (use this name – all lower case), and *nothing* else. Your submission will be automatically run to determine the functionality mark. If you submit a file with a different name, the tests will **fail** and you will get **zero** for functionality. Do **not** submit *any* sort of archive file (e.g. zip, rar, 7z, etc.).

Upload an initial version of your assignment *at least* one week before the due date. Do this even if it is just the initial code provided with the assignment. If you are unable access Gradescope, make a post on Edstem *immediately*. Excuses, such as you were not able to login or were unable to upload a file will not be accepted as reasons for granting an extension.

When you upload your assignment it will run a set of the functionality autograder tests on your submission. It will show you the results of these tests. It is your responsibility to ensure that your uploaded assignment file runs and that it passes the tests you expect it to pass.

Late submission of the assignment will result in a deduction of 10% of the maximum possible mark per 24 hours from the deadline for up to 3 days. *After 3 days (72 hours), you will receive a mark of zero*. A one-hour grace period will be applied to the due time, after which time (16:00) your submission will be considered officially late and will receive a mark of 0. Do not wait until the last minute to submit your assignment, as the time to upload it may make it late. Multiple submissions are allowed and encouraged, so ensure that you have submitted an almost complete version of the assignment well before the submission deadline of 15:00.

Plagiarism and Referencing AI Use

This assignment must be your own individual work. By submitting the

assignment, you are claiming it is entirely your own work. You may discuss general ideas about the solution approach with other students. Describing details of how you implement a function or sharing part of your code with another student is considered to be **collusion** and will be counted as plagiarism. You **must not** copy fragments of code that you find on the Internet to use in your assignment.

Artificial Intelligence (AI) and Machine Translation (MT) are emerging tools that may support you in completing this assessment task. You may appropriately use AI and/or MT in completing your assessment tasks. However, you **must clearly reference** any use of AI or MT in each instance. A failure to reference generative AI or MT use may constitute student misconduct under the Student Code of Conduct and result in a deduction of up to 100% of your mark. Please refer to AI Guideline that has been provided on the Ultra Blackboard page of the course.

> **Important**
>
> You **must** understand all code submitted and submit a **PDF or WORD file in English** alongside with your code submission. This file must provide detailed information about how the AI tools were used. Please refer to AI Guideline for guidance on properly documenting interactions with AI.

You are encouraged to complete *both* parts A and B of the academic integrity modules *before* starting this assignment. Submitted assignments will be electronically checked for potential cases of plagiarism.